03_clustering

Anish

2026-04-26

1. Goal of this notebook

2. Load PC scores and the original feature table

```
pc <- read_csv("../data/customer_pc_scores.csv",
               show_col_types = FALSE)
customer <- read_csv("../data/customer_features.csv",
                     show_col_types = FALSE)

dim(pc)
```

```
## [1] 4334    4
```

```
glimpse(pc)
```

```
## Rows: 4,334
## Columns: 4
## $ CustomerId <dbl> 12346, 12347, 12348, 12349, 12350, 12352, 12353, 12354, 123…
## $ PC1        <dbl> 1.3128225, 3.3557929, 0.7407239, 0.4667656, -1.9501216, 1.3…
## $ PC2        <dbl> -7.19782254, 0.17915017, 0.04036116, -2.12306976, -0.652095…
## $ PC3        <dbl> -2.8058725, 0.1402689, -0.1212519, -0.9784071, -0.5965301, …
```

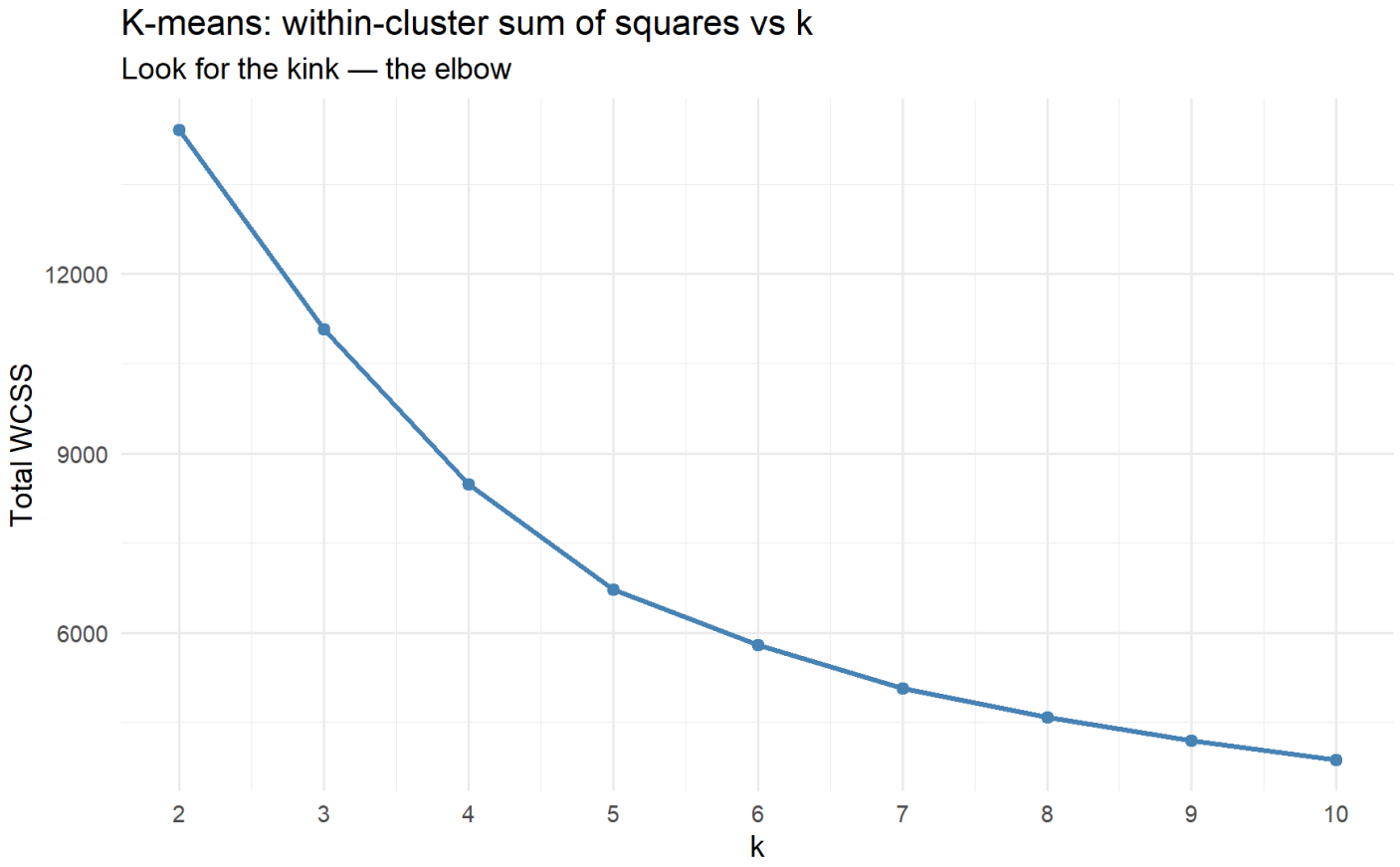
```
# Modelling matrix
Xpc <- pc |> select(starts_with("PC")) |> as.matrix()
rownames(Xpc) <- pc$CustomerId
n_pc <- ncol(Xpc)
sprintf("Clustering on %d PCs across %d customers", n_pc, nrow(Xpc))
```

```
## [1] "Clustering on 3 PCs across 4334 customers"
```

3. K-means

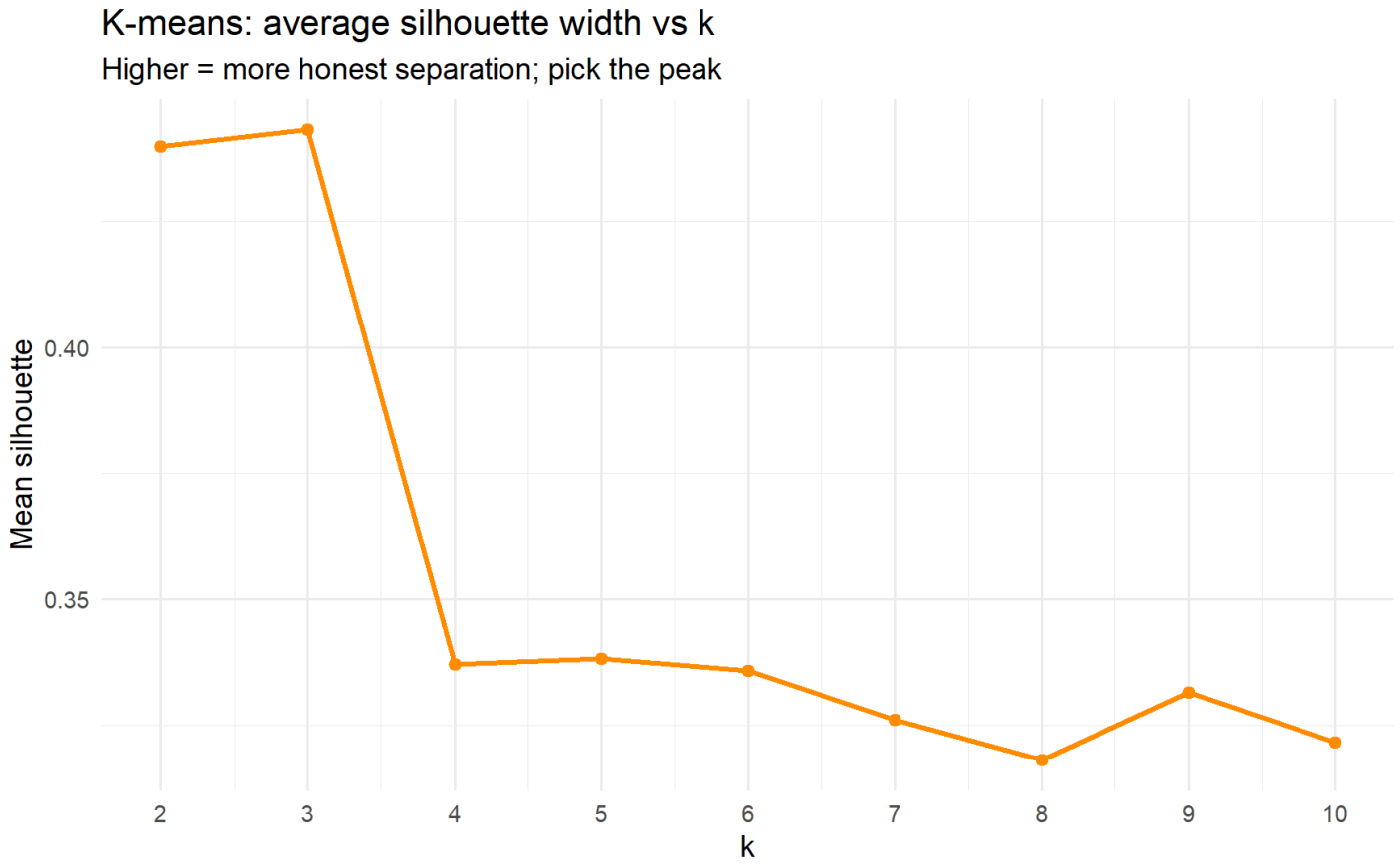
3a. Choosing K - WCSS / elbow

```
k_grid <- 2:10
wcss <- map_dbl(k_grid,
                \(k) kmeans(Xpc, centers = k, nstart = 25,
                            iter.max = 50)$tot.withinss)

elbow_df <- tibble(k = k_grid, wcss = wcss)

ggplot(elbow_df, aes(k, wcss)) +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_point(colour = "steelblue", size = 2) +
  scale_x_continuous(breaks = k_grid) +
  labs(title = "K-means: within-cluster sum of squares vs k",
       subtitle = "Look for the kink — the elbow",
       x = "k", y = "Total WCSS")
```



3b. Silhouette width

```
d <- dist(Xpc)   # used by silhouette and hclust below
sil <- map_dbl(k_grid, \(k) {
  cl <- kmeans(Xpc, centers = k, nstart = 25, iter.max = 50)$cluster
  mean(silhouette(cl, d)[, "sil_width"])
})
sil_df <- tibble(k = k_grid, silhouette = sil)

ggplot(sil_df, aes(k, silhouette)) +
  geom_line(colour = "darkorange", linewidth = 1) +
  geom_point(colour = "darkorange", size = 2) +
  scale_x_continuous(breaks = k_grid) +
  labs(title = "K-means: average silhouette width vs k",
       subtitle = "Higher = more honest separation; pick the peak",
       x = "k", y = "Mean silhouette")
```



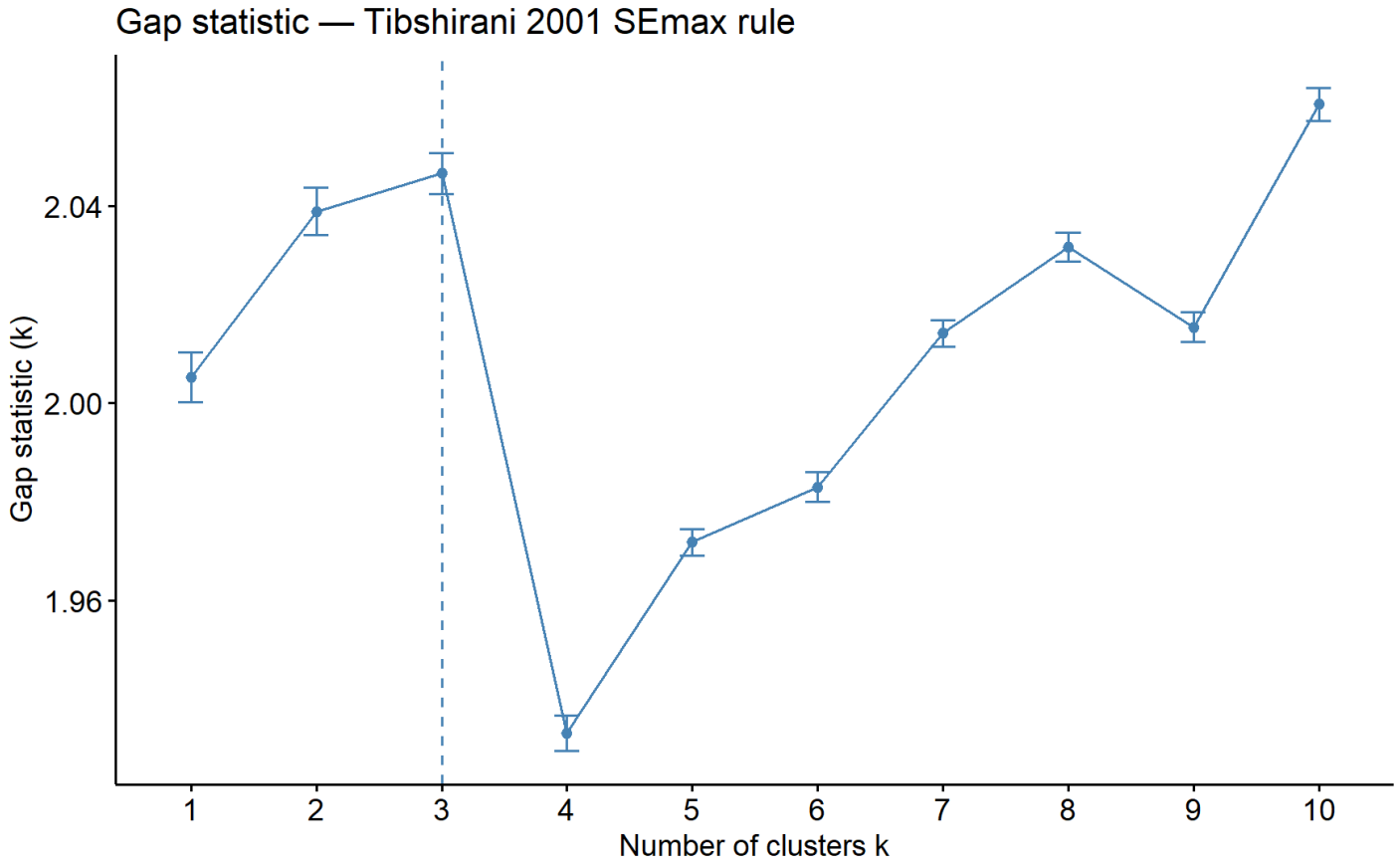
3c. Gap statistic

The most rigorous of the three (compares within-cluster dispersion to a uniform-noise reference). Slow — bumping `B` up gives smoother estimates at the cost of runtime.

```
gap <- clusGap(Xpc, FUN = kmeans, nstart = 25,
               K.max = 10, B = 50)
print(gap, method = "Tibs2001SEmax")
```

```
## Clustering Gap statistic ["clusGap"] from call:
## clusGap(x = Xpc, FUNcluster = kmeans, K.max = 10, B = 50, nstart = 25)
## B=50 simulated reference sets, k = 1..10; spaceH0="scaledPCA"
##  --> Number of clusters (method 'Tibs2001SEmax', SE.factor=1): 3
##           logW    E.logW      gap      SE.sim
##  [1,] 8.008122 10.013376 2.005254 0.005031357
##  [2,] 7.656533  9.695415 2.038883 0.004767524
##  [3,] 7.510463  9.557023 2.046560 0.004187232
##  [4,] 7.490680  9.423770 1.933090 0.003614868
##  [5,] 7.358716  9.330544 1.971828 0.002685959
##  [6,] 7.268588  9.251586 1.982998 0.002972993
##  [7,] 7.192272  9.206451 2.014180 0.002706496
##  [8,] 7.133030  9.164696 2.031666 0.002985710
##  [9,] 7.111730  9.127158 2.015428 0.003038975
## [10,] 7.031220  9.091784 2.060564 0.003282150
```

```
fviz_gap_stat(gap) +
  labs(title = "Gap statistic — Tibshirani 2001 SEmax rule")
```



3d. Pick k and fit the final model

The three diagnostics often disagree by ±1. Pick the value that's defensible under at least two and matches the substantive prior (4–6 clusters from the EDA discussion).

```
k_star <- 3  # CHANGE if your diagnostics point elsewhere

km_fit <- kmeans(Xpc, centers = k_star, nstart = 50, iter.max = 100)
km_fit$size                           # cluster sizes
```

```
## [1] 1639 1487 1208
```

```
round(km_fit$centers, 2)              # centroids in PC space
```

```
##     PC1   PC2   PC3
## 1  0.01 -0.46 -0.24
## 2 -1.97  0.24  0.12
## 3  2.42  0.33  0.18
```

```
mean(silhouette(km_fit$cluster, d)[, "sil_width"])
```

```
## [1] 0.3349896
```

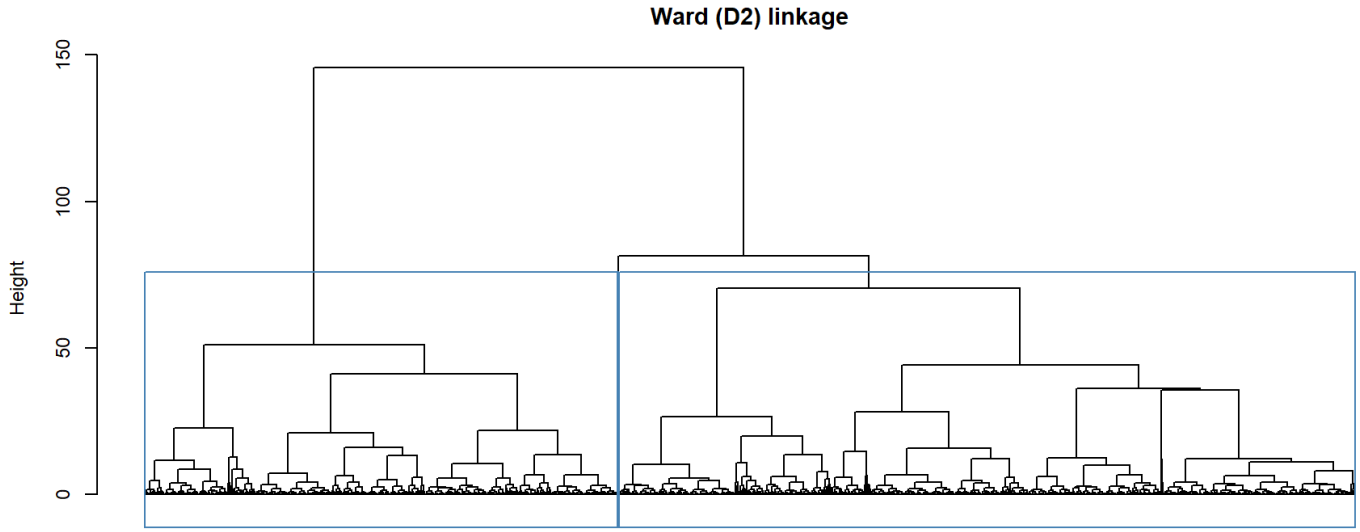
4. Hierarchical clustering

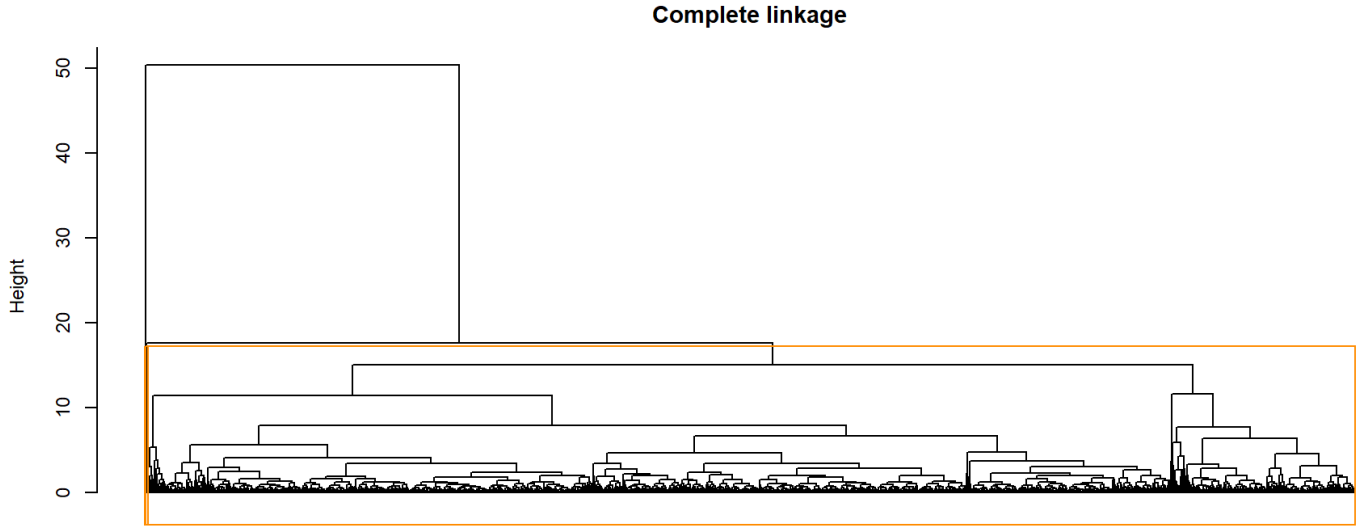
- **Ward (`ward.D2`)** — minimises within-cluster variance, lines up philosophically with k-means. Usually the strongest single choice for customer segmentation.
- **Complete** — ISLR's default in the lab; tends to give compact, similar-diameter clusters.
- **Average** — friendlier to elongated or unequal-density clusters; used here as a robustness check.

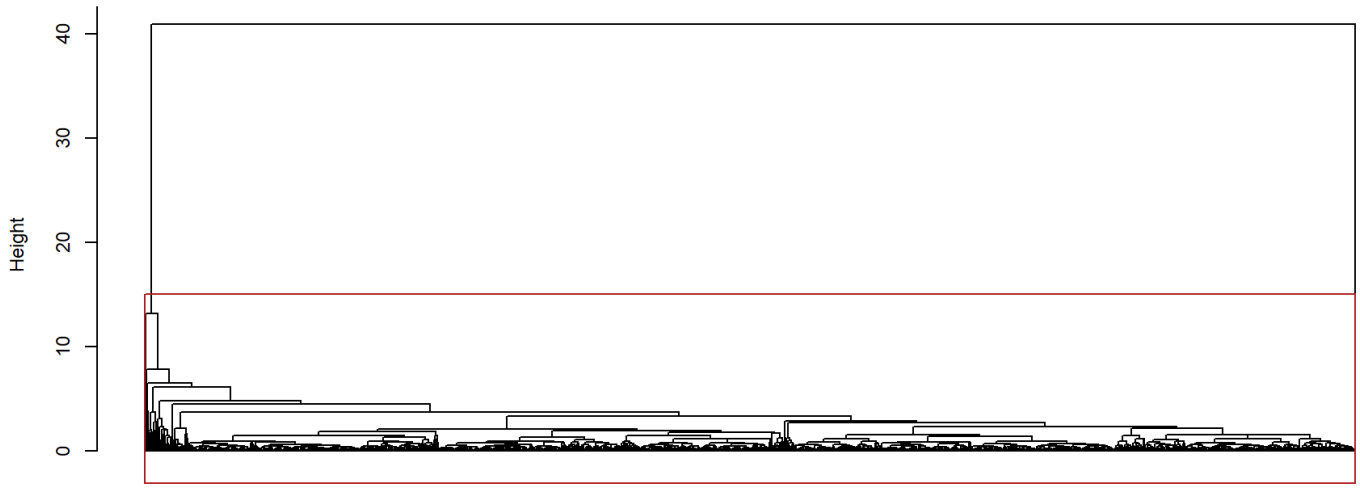
4a. Fit and plot dendrograms

```
hc_ward     <- hclust(d, method = "ward.D2")
hc_complete <- hclust(d, method = "complete")
hc_average  <- hclust(d, method = "average")
```

```
par(mfrow = c(3, 1), mar = c(2, 4, 3, 1))
plot(hc_ward,     labels = FALSE, hang = -1,
     main = "Ward (D2) linkage",     xlab = "", sub = "")
rect.hclust(hc_ward, k = k_star, border = "steelblue")

plot(hc_complete, labels = FALSE, hang = -1,
     main = "Complete linkage",      xlab = "", sub = "")
rect.hclust(hc_complete, k = k_star, border = "darkorange")

plot(hc_average,  labels = FALSE, hang = -1,
     main = "Average linkage",       xlab = "", sub = "")
rect.hclust(hc_average,  k = k_star, border = "firebrick")
```








```
par(mfrow = c(1, 1))
```

The dendrogram tells you whether `k_star` is being forced or whether the tree naturally falls into that many clusters. Ward and complete should look broadly similar; if average linkage produces one giant cluster and a handful of singletons, it's flagging a chaining problem (a real concern with retail data) — that's a finding, not a bug.

4b. Cut the tree at k* and compare to k-means

```
hc_cluster <- cutree(hc_ward, k = k_star)

table(kmeans = km_fit$cluster, hclust = hc_cluster)
```

```
##       hclust
## kmeans    1    2    3
##      1 1152  486    1
##      2 1484    2    1
##      3    2 1206    0
```

A nearly block-diagonal cross-tab means the two algorithms agree on who's in which segment (just with different label numbers). Heavy off-diagonal mass means the algorithms disagree about cluster boundaries — usually because k-means prefers spherical, equal-size clusters and hierarchical doesn't.

```
mean(silhouette(hc_cluster, d)[, "sil_width"])
```

```
## [1] 0.4323457
```

5. Stability — bootstrap Jaccard

This is the validation step that separates "real structure" from "k-means happened to land somewhere." Resample customers (with replacement), refit the clusterer, and measure how often the *original* clusters reappear. Jaccard ≥ 0.75 = "stable cluster", < 0.6 = "probably noise" (Hennig 2007).

```
boot_km <- clusterboot(
  Xpc,
  B            = 50,
  bootmethod   = "boot",
  clustermethod = kmeansCBI,
  krange       = k_star,
  seed         = 1,
  count        = FALSE
)
round(boot_km$bootmean, 3)           # one Jaccard per cluster
```

```
## [1] 0.878 0.926 0.921
```

```
boot_km$bootbrd                       # how often dissolved
```

```
## [1] 2 0 0
```

```
boot_hc <- clusterboot(
  Xpc,
  B            = 50,
  bootmethod   = "boot",
  clustermethod = hclustCBI,
  method       = "ward.D2",
  k            = k_star,
  seed         = 1,
  count        = FALSE
)
round(boot_hc$bootmean, 3)
```

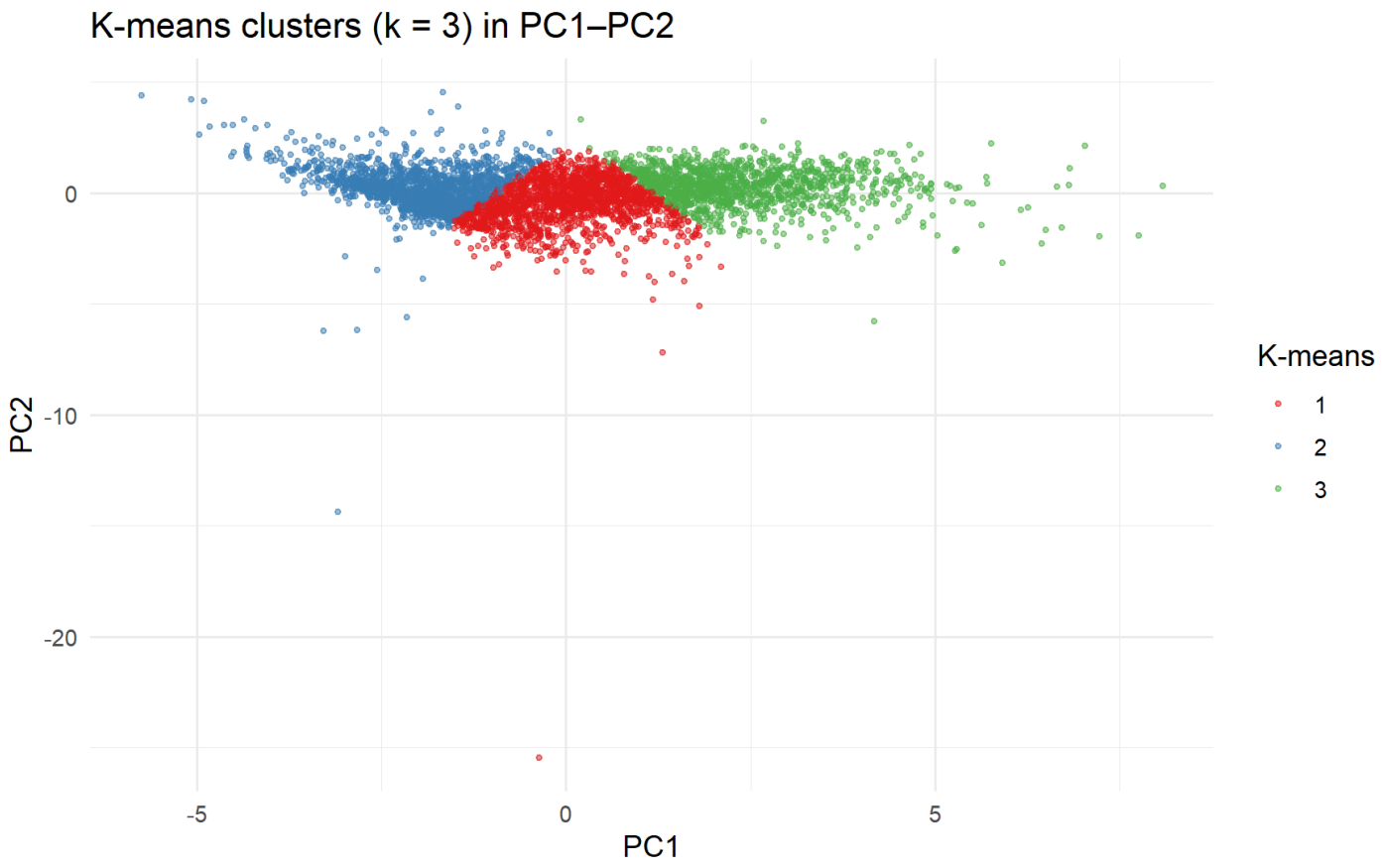
```
## [1] 0.669 0.594 0.809
```

```
boot_hc$bootbrd
```

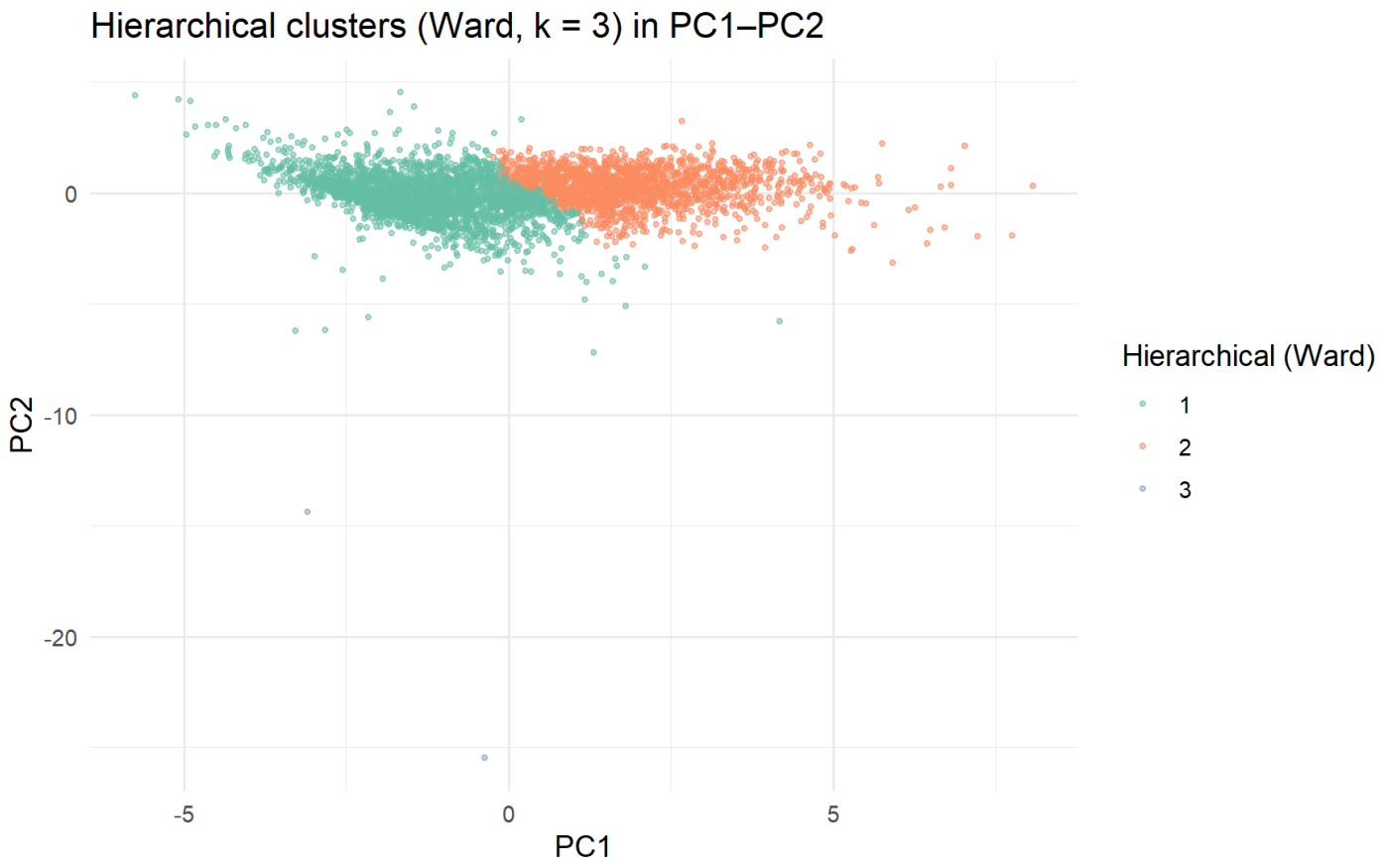
```
## [1]  6 11 10
```

6. Visualise the final clusters in PC1–PC2

```
plot_df <- pc |>
  mutate(km_cluster = factor(km_fit$cluster),
         hc_cluster = factor(hc_cluster))

ggplot(plot_df, aes(PC1, PC2, colour = km_cluster)) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_colour_brewer(palette = "Set1", name = "K-means") +
  labs(title = sprintf("K-means clusters (k = %d) in PC1–PC2", k_star))
```




```
ggplot(plot_df, aes(PC1, PC2, colour = hc_cluster)) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_colour_brewer(palette = "Set2", name = "Hierarchical (Ward)") +
  labs(title = sprintf("Hierarchical clusters (Ward, k = %d) in PC1–PC2",
                       k_star))
```




```
final_label_source <- "kmeans"   # CAN CHANGE to "hclust"

final_cluster <- if (final_label_source == "kmeans")
                   km_fit$cluster else hc_cluster

assignments <- pc |>
  select(CustomerId) |>
  mutate(cluster        = factor(final_cluster),
         cluster_source = final_label_source,
         k              = k_star)

write_csv(assignments, "../data/customer_clusters.csv")

assignments |> count(cluster, name = "customers")
```

cluster <fct>	customers <int>
1	1639
2	1487
3	1208

3 rows